Software Engineering
Final Project Guidelines

1. Executive Overview & Objective

The Software Engineering final project is designed to transition students from theoretical concepts to industrial-grade software engineering practices. Teams are required to engineer a scalable, real-world software system by strictly implementing the full Software Development Life Cycle (SDLC).

- ❖ **Coursework Weight: The project accounts for 40 marks out of the 50 total coursework marks.**

2. Team Formation

Teams must consist of **exactly three (3) students** carefully selected based on mutual understanding, reliable communication, and equal commitment. (No individual submissions, pairs, or larger groups will be permitted).

3. Project Selection & Scope

Projects must address practical, real-world problems. Proposals featuring highly realistic problem domains and clear user utility will be prioritized in evaluation. Teams are strongly encouraged to propose solutions that serve Damanhour University (e.g., systems for university administration, student services, the Faculty of Medicine, or the University Hospital).

- **Tech Stack:** **There are no restrictions** on programming languages, frameworks, libraries, or any technology. Projects may be developed as mobile applications, web applications, or desktop software.
- ❖ **Exceptional projects addressing Damanhour University needs may be considered for official institutional adoption.**

4. Requirements & Guidelines

1. Execute and document every software engineering phase, from requirement gathering to architectural design, implementation, testing, and final documentation.
2. Build a clean, well-structured system architecture designed for modularity, maintenance, and future scalability.
3. Write clean, readable code that adheres to unified coding conventions and style standards throughout the entire repository.
4. Deliver complete documentation, including a detailed SRS report, UML diagrams, ERDs, and final documentation.
5. **Every team member must actively contribute to all layers of the project** (specialized team member work is prohibited)
6. **AI tools are permitted**, but every member must thoroughly understand every line of code and be prepared to explain, refactor, or rebuild code segments live during evaluation.

5. Evaluation & Grading Criteria

- **Team-Based Grading:** Project grades reflect overall team performance and cohesion.
- **Shared Accountability:** Unequal workload distribution or failure of individual members to demonstrate mastery of the codebase will severely impact the entire team's grade. A high-quality project delivered by only one active member while others remain passive will result in a poor overall evaluation for the team.
- **Oral Examination Protocol:** During the final project evaluation, instructors may test individual understanding by asking line-by-line code explanations, requesting live refactoring, or deleting specific code segments and asking team members to recreate them on the spot.